VÕ TIẾN
Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

Cấu Trúc Dữ Liệu và Giải Thuật (DSA)

DSA - HK243

Cuối Kì

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

TP. HỒ CHÍ MINH, THÁNG 7/2025

Dùng cho 10 câu sau

```
template <typename T>
class DoublyLinkedList
{
private:
    struct Node
    {
        T data;
        Node *prev;
        Node *next;
        Node() : prev(nullptr), next(nullptr) {}
        Node(const T &val, Node *prev = nullptr, Node *next = nullptr)
            : data(val), prev(prev), next(next) {}
    };

    Node *head = nullptr;
    Node *tail = nullptr;
    int length = 0;

public:
    DoublyLinkedList();
    ~DoublyLinkedList();

    void insertAtHead(T data);
    void insertAtTail(T data);
    void insertAt(int index, T data);
    void deleteAt(int index);
    T &get(int index) const;
    int indexOf(T item) const;
    bool contains(T item) const;
    int size() const;
    void reverse();
    string toString(string (*convert2str)(T &) = nullptr) const;
};
```

1. Điền code vào đoạn còn trống **CODE 1 và CODE 2**

```
void insertAtHead(T data) {
    head = /* CODE 1 */;

    if (length == 0) {
        tail = head;
    } else {
        /* CODE 2 */
    }
    ++length;
}
```

a) `head = new Node(data, nullptr, head); head->next->prev = head;`
b) `head = new Node(data); head->next = head;`
c) `head = new Node(data, nullptr, head); head->prev = head->next;`
d) `head = new Node(data); head->prev = head;`

Đáp án: A

2. Điền code vào đoạn còn trống **CODE 1 và CODE 2**

```
void insertAtTail(T data) {
    tail = /* CODE 1 */;
```

```

    if (length == 0) {
        head = tail;
    } else {
        /* CODE 2 */
    }
    ++length;
}
```

a) `tail = new Node(data, tail, nullptr); tail->prev->next = tail;`
b) `tail = new Node(data); tail->prev = tail;`
c) `tail = new Node(data, tail, nullptr); tail->next = tail->prev;`
d) `tail = new Node(data); tail->next = tail;`

Đáp án: A

3. Điền code vào đoạn còn trống **CODE 1 và CODE 2**

```
Node* getNode(int index) const {/* TODO */}
void insertAt(int index, T data) {
    /* TODO */
    Node* cur = getNode(index);
    Node* newNode = /* CODE 1 */;
    cur->prev->next = newNode;
    /* CODE 2 */
    ++length;
}
```

a) `newNode = new Node(data, cur->prev, cur); cur->prev = newNode;`
b) `newNode = new Node(data); cur->next = newNode;`
c) `newNode = new Node(data, cur, cur->prev); cur->next = newNode;`
d) `newNode = new Node(data); cur->prev = newNode;`

Đáp án: A

4. Điền code vào đoạn còn trống **CODE 1 và CODE 2**

```
Node* getNode(int index) const {/* TODO */}
void insertAt(int index, T data) {
    /* TODO */
    Node* cur = getNode(index - 1);
    Node* newNode = /* CODE 1 */;
    /* CODE 2*/
    cur->next = newNode;
    ++length;
}
```

a) `newNode = new Node(data, cur, cur->next); cur->next->prev = newNode;`
b) `newNode = new Node(data); cur->next = newNode;`
c) `newNode = new Node(data, cur->next, cur); cur->next = newNode`
d) `newNode = new Node(data); newNode->next = cur->next;`

Đáp án: A

5. Trong hàm `deleteAt(int index)`, giả sử đã kiểm tra `index` hợp lệ, đoạn code nào xử lý đúng khi xóa node ở đầu hoặc cuối danh sách liên kết đôi không dùng kiểm tra con trỏ `nullptr`, mà dựa vào chỉ số `index` và `length`?

```cpp
Node* getNode(int index) const {/* TODO */}
void deleteAt(int index) {
    /* TODO */
    Node* cur = getNode(index);

    if (/* CODE 1 */) {
        head = cur->next;
        if (head != nullptr)
            head->prev = nullptr;
    } else {
        cur->prev->next = cur->next;
    }

    if (/* CODE 2 */) {
        tail = cur->prev;
        if (tail != nullptr)
            tail->next = nullptr;
    } else {
        cur->next->prev = cur->prev;
    }

    /* TODO */
}
```

a) `index == 0` và `index == length - 1`
b) `index == 0` và `index == length`
c) `index == 1` và `index == length`
d) `index == 1` và `index == length - 1`

Đáp án: A

6. Độ phức tạp tính toán (time complexity) của hàm `findLastIndexOf` sau là bao nhiêu?

```cpp
int findLastIndexOf(const DoublyLinkedList<char>& list, char target) {
    int lastIndex = -1;
    int n = list.size();
    for (int i = 0; i < n; ++i) {
        if (list.get(i) == target) {
            lastIndex = i;
        }
    }
    return lastIndex;
}
```

a) `O(n)`
b) $O(n^2)$
c) $O(\log n)$
d) `O(1)`

Đáp án: B

7. Khi thực hiện hàm `reverse()` trên một `DoublyLinkedList`, đoạn code nào sau đây là cần thiết để đảo ngược liên kết giữa các node?

```cpp
void reverse() {
    Node* cur = head;
    Node* tmp = nullptr;
    while (cur) {
        /* CODE */
    }
    Node* swap = head;
    head = tail;
    tail = swap;
}
```

a) `tmp = cur->prev; cur->prev = cur->next; cur->next = tmp; cur = cur->prev;`
b) `tmp = cur->next; cur->next = cur->prev; cur->prev = tmp; cur = cur->next;`
c) `cur->prev = cur->next; cur->next = cur->prev; cur = cur->prev;`
d) `cur->next = tmp; tmp = cur->prev; cur = cur->prev;`

Đáp án: A

8. Lợi ích chính khi sử dụng `dummy node` (node giả) trong cấu trúc `DoublyLinkedList` là gì?

a) Giúp đơn giản hóa việc xử lý các trường hợp thêm, xóa ở đầu hoặc cuối danh sách, tránh phải kiểm tra `nullptr`.
b) Giúp tiết kiệm bộ nhớ so với danh sách không dùng dummy node.
c) Tăng tốc độ truy cập ngẫu nhiên trong danh sách liên kết.
d) Giúp giảm độ phức tạp thời gian truy cập phần tử xuống `O(1)`.

Đáp án: A

9. Điền đoạn code còn thiếu để tìm điểm giữa của danh sách liên kết dùng con trỏ nhanh và chậm:

```
Node* findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (/* CODE 1 */) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

a) `fast != nullptr && fast->next != nullptr`
b) `fast->next != nullptr`
c) `fast != nullptr`
d) `slow != nullptr && fast != nullptr`

Đáp án: A

10. Điền code vào đoạn còn trống **CODE 1 và CODE 2**

```
Node* mergeSortedLists(Node* left, Node* right) {
    // Điều kiện dừng đệ quy
    if (left == nullptr) return right;
    if (right == nullptr) return left;

    // So sánh giá trị của hai nút
    if (left->data <= right->data) {
        /* CODE 1*/
        if (left->next != nullptr) {
            /* TODO */
        }
        return left;
    }
    /* TODO */
    if (right->next != nullptr) {
        /* CODE 2*/
    }
    return right;
}
```

a) `left->next = mergeSortedLists(left->next, right); right->next->prev = right;`

b) `left->next = mergeSortedLists(left->next, right); right->prev->prev = right;`
c) `left->next = mergeSortedLists(left->prev, right); right->next->prev = right;`
d) `left->next = mergeSortedLists(left->prev, right); right->prev->prev = right;`

Đáp án: A